



Reactor

OVERVIEW

What's new in model router?



Hosted By: Lee Stott

Aug 24, 2026 · 1:30-2:30 PM ET



Sanjeev Jagtap

Principal Lead, Core AI / Microsoft

What is the model router?

A purpose-built, trained **language model** that analyzes each prompt in real time and routes it to the **best-fit LLM**.



Auto-navigates the Foundry model catalog

Picks the best model based on task complexity and your cost/quality preferences.



Routing modes & model subsets

Balanced, Cost, or Quality. Restrict the pool to specific models for compliance or budget.



Foundry Agent Service support

Same routing benefits in agentic, multi-step tool-calling workflows.



Enterprise reliability of Azure

Built-in security, automatic failover, observability, and data-zone boundaries.

Tune the cost-quality tradeoff

Three modes adjust the quality band — the tolerance for routing to a cheaper model. Even Quality mode doesn't blindly send everything to the most expensive option.



Cost

Wide quality band (~5-6%)

Aggressively favors cheaper models. Nano-class handles the vast majority of traffic.



Balanced

Narrow band (~1-2%)

Picks the most cost-effective eligible model. Spreads traffic across tiers by difficulty.

DEFAULT



Quality

Highest-scoring model

Ignores cost. Frontier models for hard prompts; cheaper still chosen when equivalent.

Up to **76%**

cost savings when compared to the most expensive model.
Based in internal data

A four-stage routing pipeline

A compact ML classifier — not an LLM — predicting which model performs best for each prompt at millisecond overhead.

01



02



03



04

Prompt analysis

Prompt complexity, reasoning depth, Task type, tool/ functional calling needs, and context length and structure—into an internal representation of intent and difficulty.

Quality estimation

Prompts that any capable model can handle well vs. prompts that demand deeper reasoning, nuanced judgment. The difficulty-aware routing allows model router to save costs without sacrificing quality.

Purposeful selection

Balanced mode is default. In Cost mode, most traffic routes to smaller, cheaper escalating to larger models only when needed. In Quality mode, frontier and high-capability models handle most traffic.

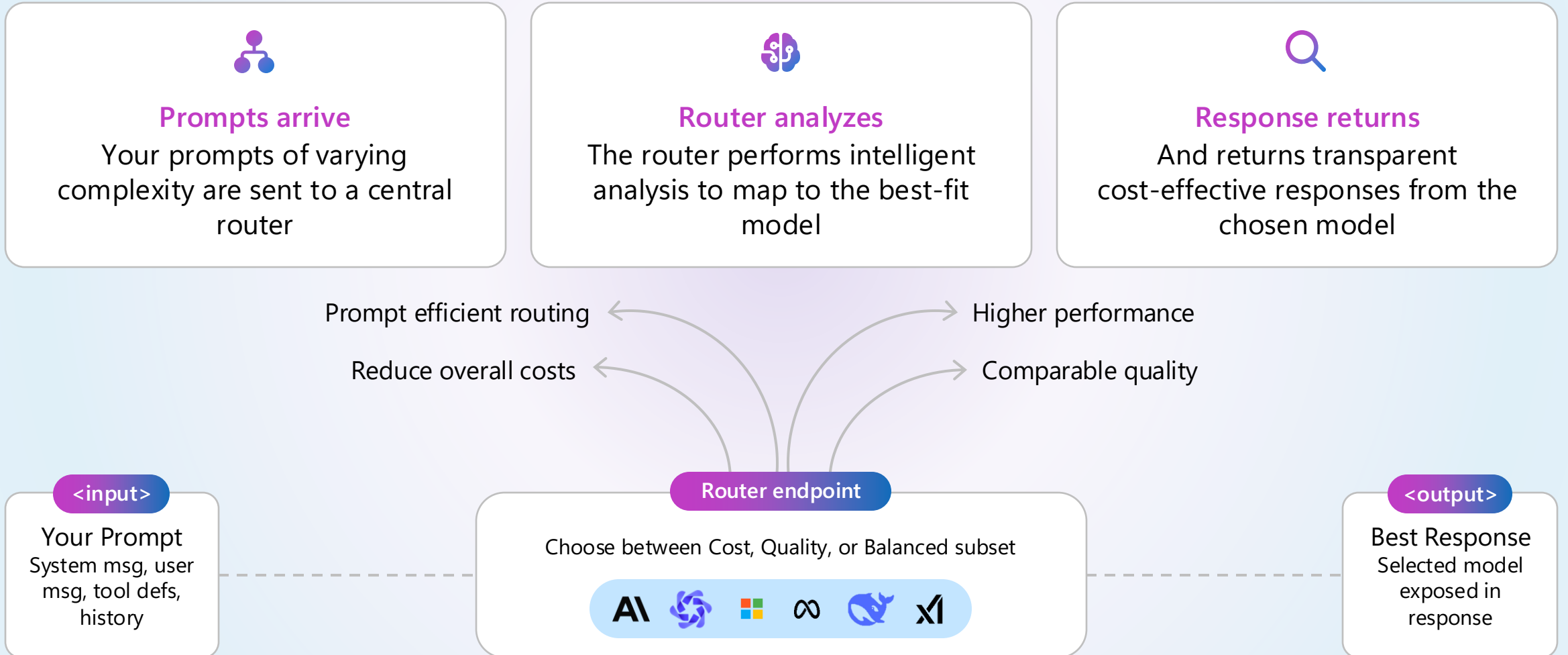
Dispatch

Forwards the prompt to the selected model. End-to-end routing overhead: milliseconds — negligible vs. LLM inference. Prompts are never stored.

Trained on hundreds of thousands of examples · Dozens of domains ·
Adversarial + tool-calling coverage · Prompts are never stored

Automatic model selection in real time

Automatically select optimal models for a specific task or query in real-time.



Supported models | Global and Data Zones

1 deployment, 1 endpoint, 27 models

NEW MODELS THIS MONTH

■ 4 added

■ 5 deprecated

■ OPENAI

■ DEEPSEEK

■ ANTHROPIC

■ META

■ XAI

Model Subset & Compliance Control

●● Custom Deploy

You decide which models participate — your compliance gate

✓ Approved for routing

gpt-5

gpt-5-mini

gpt-4.1

gpt-4.1-mini

o4-mini

✗ Excluded

grok-4

DeepSeek-V3.2

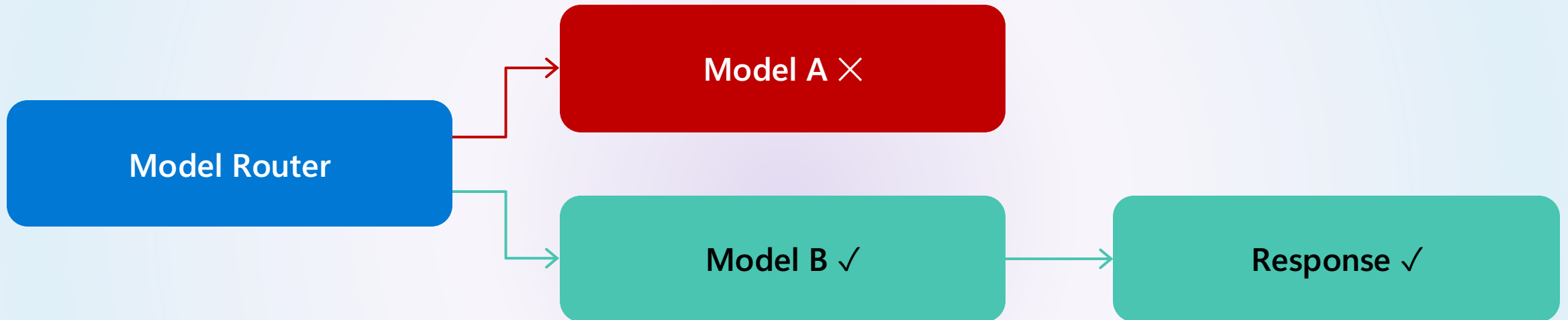
Llama-4

Why Model Subset?

- Only approved models
- New models excluded by default
- Explicit opt-in for additions
- Configure via Portal or REST API
- 2+ models enables failover
- Auto-recalibrates routing

Automatic Failover

Built-in resilience — transient issues never disrupt your application



Why it matters

This resolves the tension: customers get the full breadth of the model pool with platform-grade reliability. The router becomes self-healing — endpoint instability is absorbed transparently.

How it works

- ✓ Zero configuration — failover enabled by default
- ✓ Transparent to your app — response arrives normally
- ✓ Routing mode still applies during failover
- ✓ Model subset = your fallback pool (select 2+ models)

Policy enforcement across the routing stack

Governance applies at the router and cascades to every model in your selected subset

01 Router Governance

Azure Policy definitions apply to Model Router as a single deployment point. One policy controls access, regions, and compliance for the entire routing endpoint.

Applies across portal, CLI, SDK, and ARM/Bicep — a single rule governs the entire routing surface.

02 Subset Model Compliance

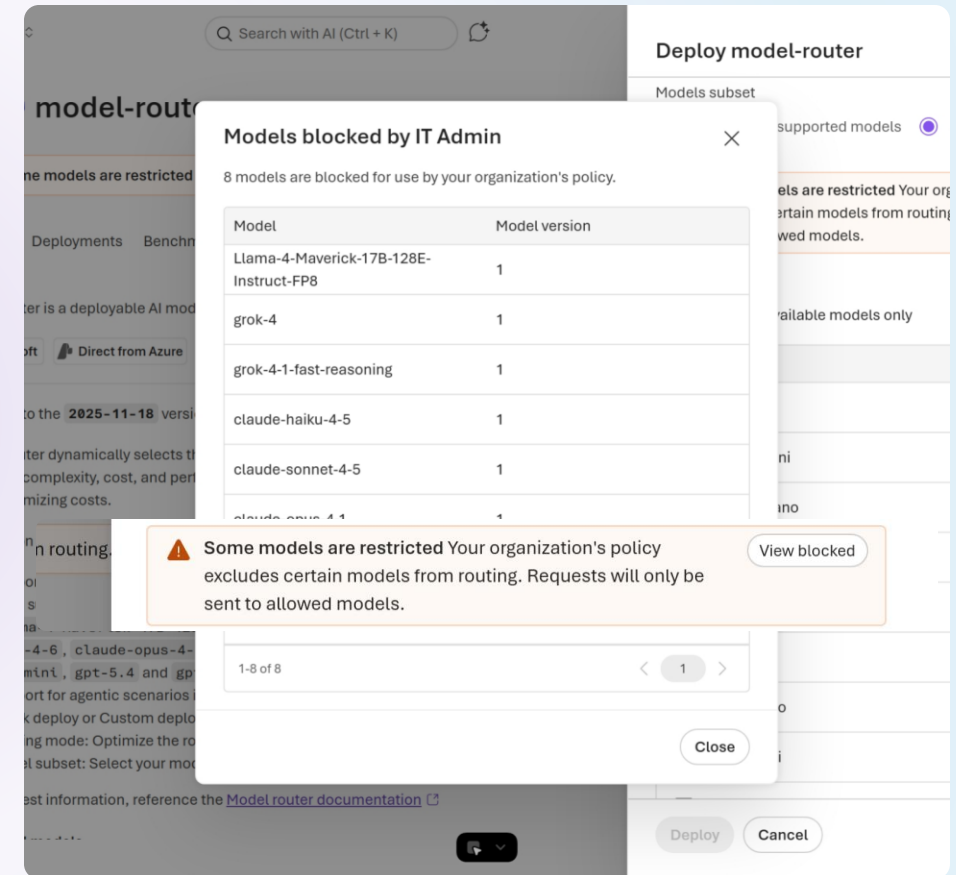
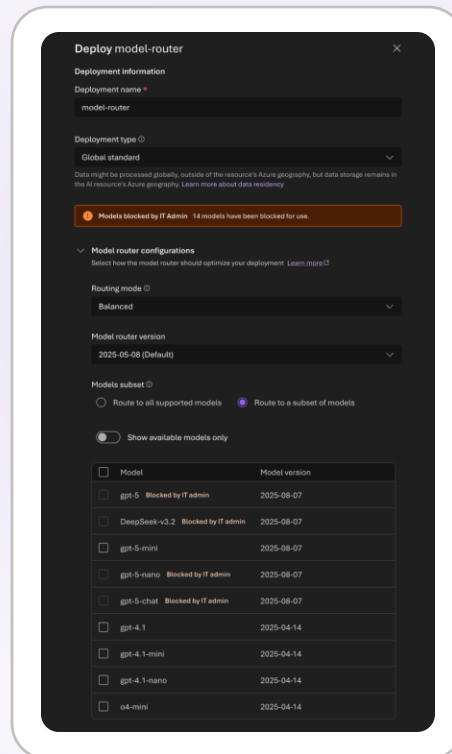
Every model in your configured subset inherits and enforces the same policy constraints set at the router level.

Content filtering, data residency, and access controls are honored at the individual model level — not just at the routing layer.

03 Enterprise Impact

60%+ cost and latency savings with full compliance — no tradeoff between performance and governance.

One deployment surface for all 28 models across 6 providers — consistent audit trail and compliance posture.



Policies cascade from router to every model in your subset

Benchmark the router on your prompts

An open-source Python toolkit that closes the gap left by built-in Foundry benchmarks — evaluate model router against any baseline on **quality, cost, and latency**.

```
# Dry-run - validate config and dataset, no API calls
python scripts/run_eval.py --dry-run

# Full evaluation (default: 10 sample prompts)
python scripts/run_eval.py

# Custom dataset / sample size (JSONL, CSV, or database)
python scripts/run_eval.py --dataset my_prompts.jsonl --sample-size 100
python scripts/run_eval.py --dataset my_prompts.csv
python scripts/run_eval.py --dataset "sqlite:///prompts.db?table=prompts"
```

One command, three outputs



HTML dashboard

Self-contained, interactive — open in any browser.



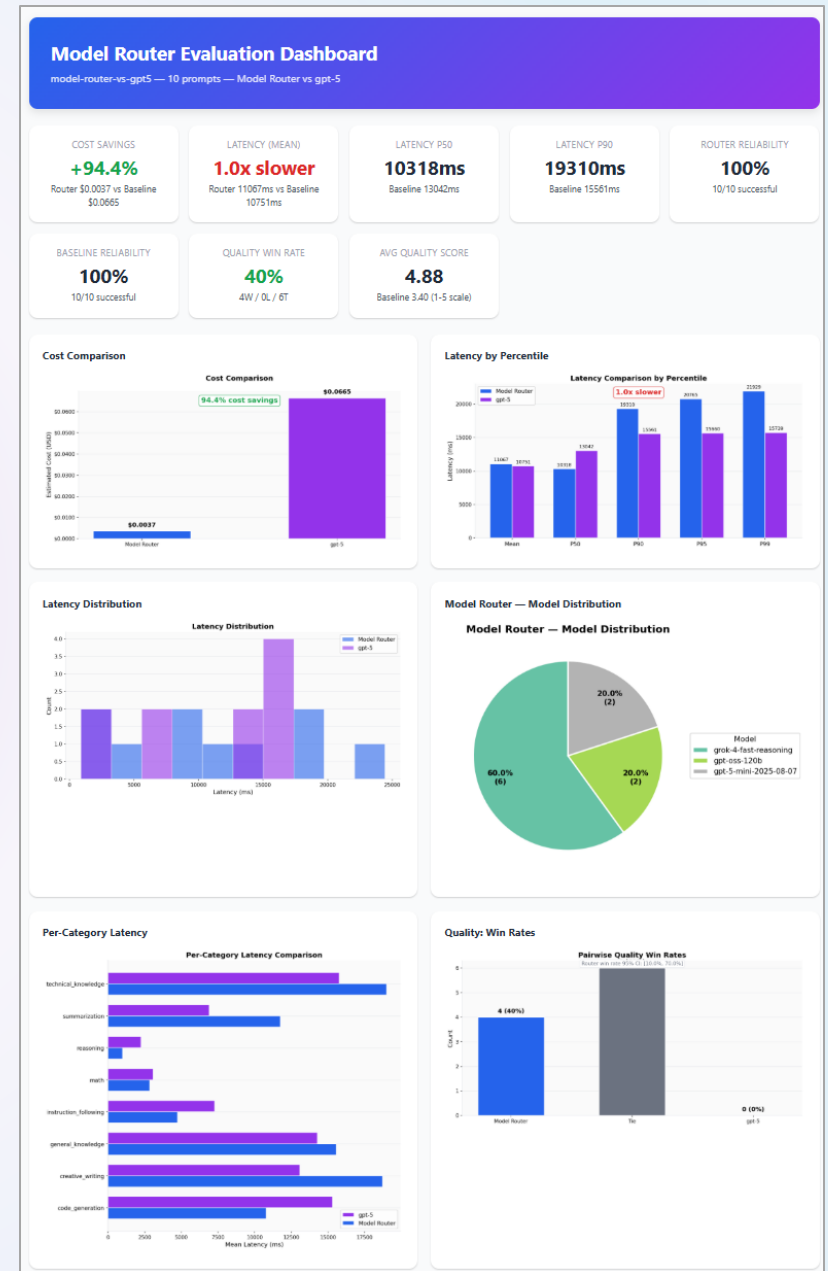
Markdown report

Shareable summary for PRs, docs, decision logs.



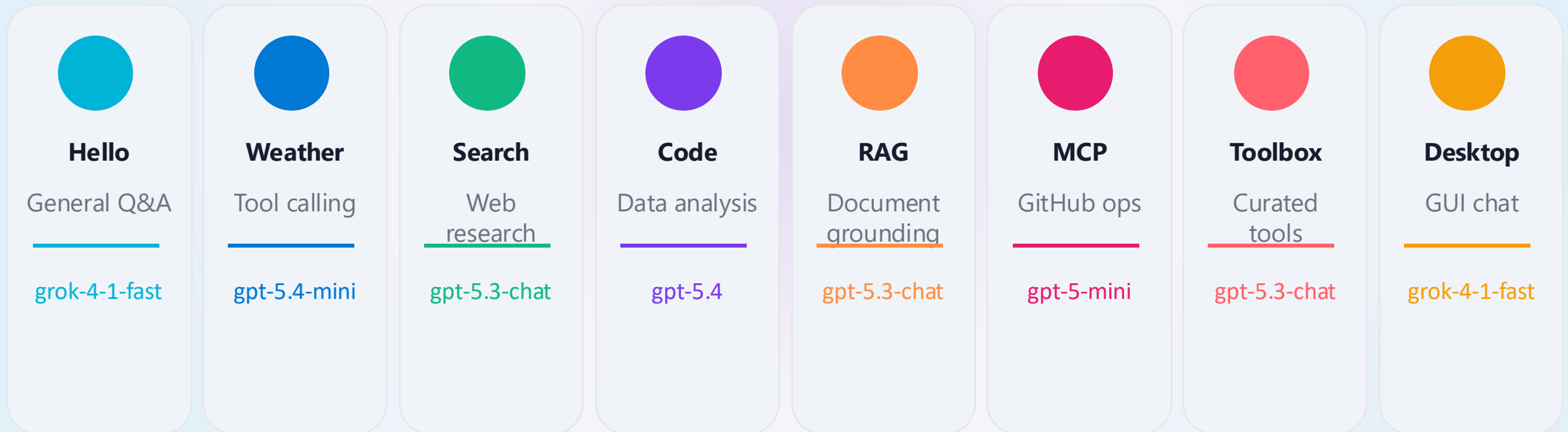
Machine-readable results

JSON output for CI/CD pipelines and custom analysis.



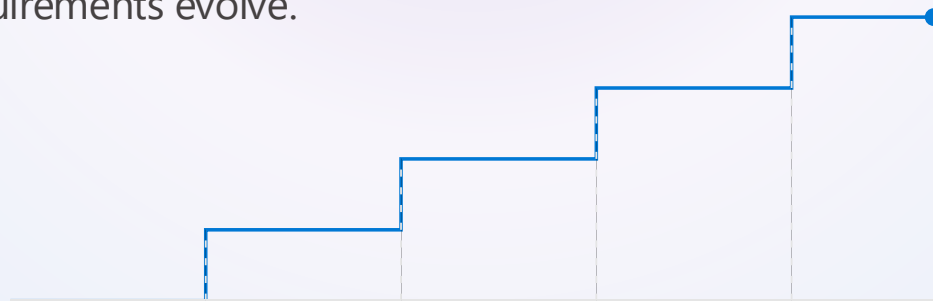
Multiple agents, one model deployment

● ● Overview: Foundry-Agent-Lab *No model pinning, no routing hints — the only variables are tools, system prompt, and query.*



What is hill climbing?

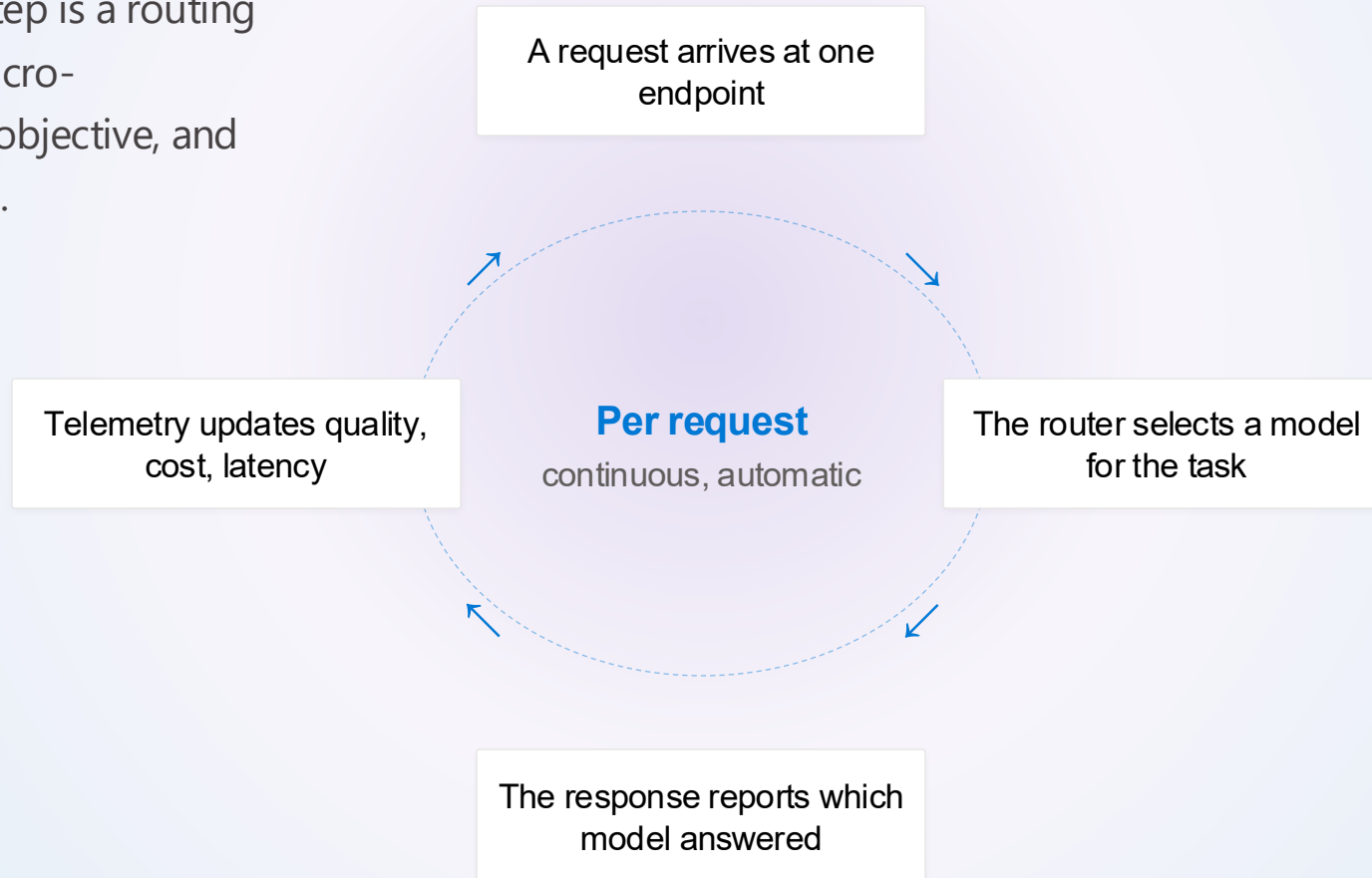
Across Microsoft, "hill climbing" has become shorthand for how real AI progress happens: not in one dramatic leap, but through a disciplined optimization loop. Whether improving frontier models through better data, compute, and evaluation, or refining application performance across quality, latency, and cost, the underlying principle is the same: progress is not a one-time decision. It's a continuous cycle of measurement, learning, and improvement. Model router brings that philosophy to the selection layer, giving teams a tool not only for routing requests, but also for evaluating tradeoffs, understanding workload behavior, testing optimization strategies, and building a repeatable optimization loop as models, applications, and requirements evolve.



Hill climbing with the model router

The same loop, closed automatically:

At the selection layer, a step is a routing decision. Each one is a micro-optimization against the objective, and each one is instrumented.



Three ways to run the hill climbing loop

A / B TESTING

Compare two router configurations

Send representative traffic to two deployments that differ by routing mode or model subset, then compare quality, cost, and latency.

MODEL DECOMPOSITION

Break one model into a system

Use routing results to decompose a single-model application into a multi-model or multi-agent design.

CONTINUOUS ROUTING

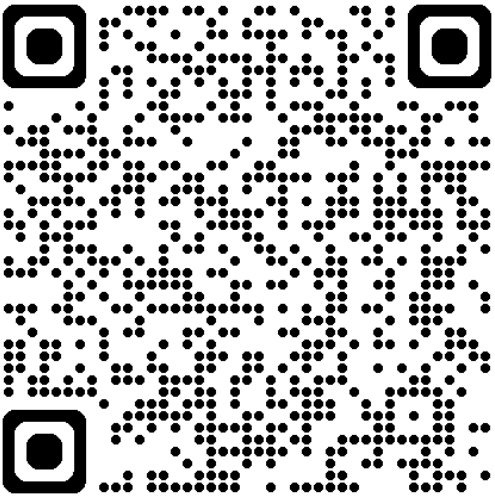
Keep the router in production

Model selection stays a per-request decision instead of a per-project one.

Each pattern turns model choice into a measured, repeatable process rather than a fixed decision.

What's New in the
model router in
Foundry Models?

Explore the release
notes for details and
resources to use



The model router in Microsoft Foundry is kept up to date automatically – you get access to the latest features transparently. Bookmark and visit this page to get the latest release notes on new models and capabilities supported



Updates to the model router in Foundry Models

MICROSOFT FOUNDRY BLOG 5 MIN READ

Model router updates: new regions, a refreshed model pool, and understanding the hill climb

 **sanjeev_jagtap**  MICROSOFT
Aug 21, 2026

Across Microsoft, "hill climbing" has become shorthand for how real AI progress happens: not in one dramatic leap, but through a disciplined loop. [Microsoft AI defines the hill climb](#) as an organization that continuously improves, cycle after cycle, through more compute, better data, and sharper evaluation. [Reinforcement fine-tuning in Foundry](#) defines it as improving the deployable model package one measured step at a time across quality, latency, and cost. Different altitudes, same premise: **progress is not a one-shot decision. It's a loop.**

For most teams, the decision of what model to use when is made manually or with custom routing tools. A developer picks a model based on benchmarks, familiarity, or the last launch that made headlines, ships it, and revisits the choice only when something breaks. In an ecosystem where the frontier moves monthly, that decision goes stale fast. **Model router in Foundry Models brings the hill climb to the selection layer.**

Visit: <https://aka.ms/modelrouter/updates>



Reactor

DEEP-DIVE

Can routing save money without losing quality?

A live, reproducible Model Router experiment

deploy · measure · compare · iterate

Workshop: <https://aka.ms/model-mastery/model-router>



Lee Stott

Principal Cloud Advocate / Microsoft

The trap

Lower cost is useful only when the answer is still correct and safe.

The tempting shortcut

“Send every request to the cheapest model.”

That may reduce the invoice, but it can also increase incorrect answers, retries and operational risk.

Why it fails

A wrong answer can be expensive to investigate or reverse, especially for account security, payment and policy questions.

The real question

Can we lower cost and latency without reducing policy compliance or account-security quality?

COST

LATENCY

QUALITY

SAFETY

The scenario · Northstar Devices

A synthetic, privacy-safe support workload with live routing evidence.

Carrier outage

High volume, repetitive customer questions. Speed matters, but the assistant still needs accurate status and guidance.

Duplicate-payment bug

Billing cases need precise classification and correct next steps. A wrong answer creates more support work.

Phishing campaign

Security cases are high risk. The assistant must recognise the threat and follow the approved response.

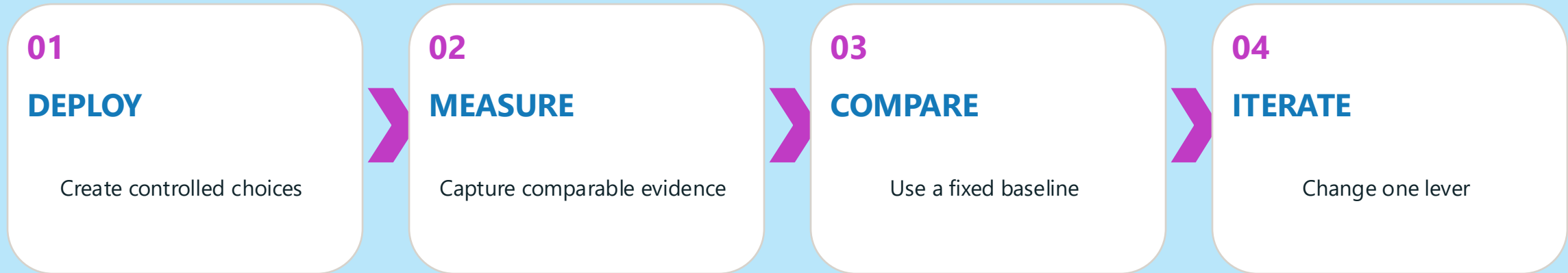
Your role

Choose the model-selection policy for the support assistant. Leadership wants lower cost and latency. You are accountable for mishandled security cases.

Synthetic prompts · Live routing · Live tokens · Live latency

The method · one repeatable loop

Never promote a change unless it clears the quality and safety floor.



Hill climbing

Keep the current best option. Test one controlled change. Promote it only when the evidence shows an improvement and every required constraint still passes.

Lab 0 · three controlled arms

A fair comparison changes one lever at a time.

A · Fixed baseline

Deployment: baseline-gpt

One fixed GPT model with no routing.

Purpose: the benchmark that every router must beat.

B · GPT-family router

Deployment: router-gpt-family

Router restricted to an approved GPT-family allowlist.

Purpose: test whether routing inside one family adds value.

C · Open-weight router

Deployment: router-open-weight

Same router configured for an open-weight model family.

Purpose: test a different eligible model pool.

Between B and C, the only lever that changes is the eligible model family.

Lab 0 · trust, but verify

Deployment automation should confirm reality, not just submit a request.

01 · Resolve the model

Confirm that the catalog model is available in the chosen region and supports the required deployment type before deploying.

02 · Read deployment back

Verify provisioning state, model version, SKU, capacity and routing subset. This catches silent drift or partial deployment.

03 · Record the result

Write the real deployment names into .env so notebooks and later runs use the same tested configuration.

The script does not create your account, permissions or quota. Those prerequisites must already exist.

Lab 1 · qualify one live router

A smoke test asks whether the router is credible enough to become a candidate.

Test set

20 synthetic cases

12 core cases

8 challenge cases

No customer data.

Task coverage

Classification

Policy retrieval

Summarisation

Constrained reasoning

Evidence per call

Underlying model selected

Tokens used

Latency

Response

Quality indicators

A smoke test is an early warning system, not a production sign-off.

Lab 1 · the honest evidence

12-call point-in-time run · router-gpt-family

3

underlying models
used

650

input tokens

4,672

output tokens

36s

p50 latency

~79s

p95 latency

58%

quality pass rate

Models selected: gpt-5-mini · gpt-5.4-mini · gpt-5.6-luna

This is not a clean win, and that is the point.

Lab 1 · averages lie, slices do not

Break the score down by task type and risk level.

Classification · strong

F1 = 1.0 on all three cases.

The task was easy enough for efficient models, and the answers were correct.

Summarisation and reasoning · weak

F1 fell to 0.19 and 0.31 on contrasting cases.

The selected model was not capable enough for those prompts.

Candidate gates

COMPLETE REQUESTS

CLASSIFY QUEUES

≥ 80% QUALITY

HIGH-RISK PASS

Tokens are a cost proxy here. The promotion decision uses a dated pricing snapshot.

Lab 1 · evidence you can share

Privacy by design keeps the decision evidence and removes the sensitive content.

REMOVED**Prompt and response text****KEPT****Selected route and model****KEPT****Input and output tokens****KEPT****Latency and evaluation scores**

The evidence you need, without the customer data you do not.

Lab 2 · the hill climb

Lab 1 qualifies a candidate. Lab 2 runs the decision experiment.

A versus B

Does routing within the GPT family beat the fixed-model baseline?

If B cannot beat A, routing has not yet earned promotion.

B versus C

Does changing the eligible family improve cost, latency or policy outcomes while preserving every floor?

Two non-negotiable rules

- 1 Independent judge · no model grades its own answer
- 2 Hold everything else constant · dataset, baseline, judge, pricing, region, mode, concurrency, retries and ordering

Lab 2 · the decision gate

A candidate is promoted only after it passes every required floor.

Measure

Quality and safety scores
Tokens and dated price
p50 and p95 latency
Errors and retries
Selected-model distribution

Decide

First prove B beats A.

Promote C only if it clears the same quality and safety floors and improves the agreed objective.

Repeat

If the candidate fails, keep the better arm.
Change one lever, run again and document the outcome.

20 prompts = smoke test · 100+ prompts = stronger promotion evidence

Take home · three transferable lessons

These practices apply even if you never use this repository.

01 · Use constraints

Frame the goal as: “cheaper without losing quality or safety”. The word “without” turns a wish into a testable decision rule.

02 · Slice the evaluation

Report core, challenge and high-risk groups separately. An overall average can hide a failure that matters.

03 · Keep a baseline

Compare routers with a fixed baseline, an independent judge, controlled arms and a dated pricing snapshot.

Evidence you can defend at a change-review board, not a hunch.

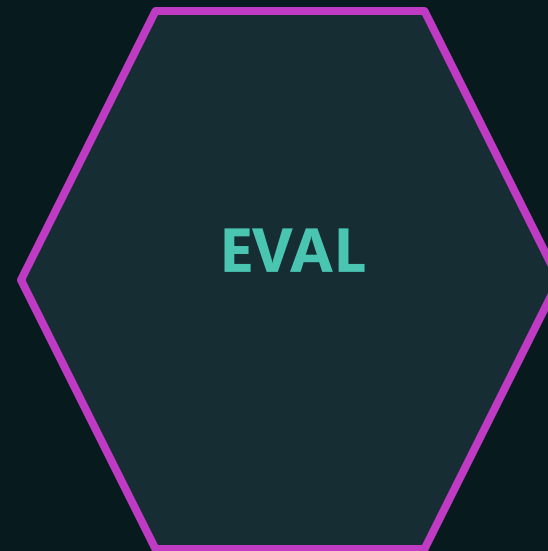
Try it

Run the controlled experiment against your own Microsoft Foundry project.

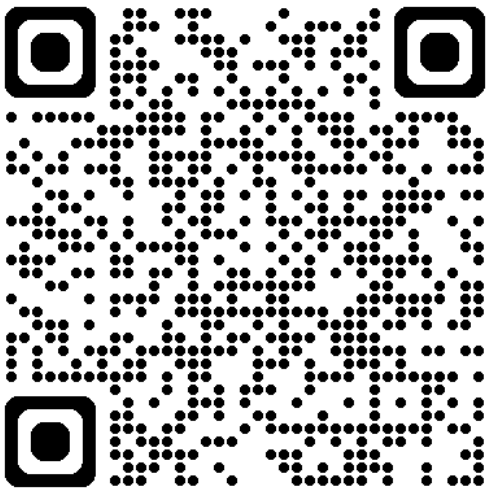
- 1 Deploy the three arms
- 2 Run the smoke test
- 3 Review the redacted scorecard

Workshop: <https://aka.ms/model-mastery/model-router>

Model Router concepts · How-to guidance · Auto Evaluation toolkit



Benchmark **model router** against a baseline model for **quality, cost, latency** — then decide whether the router is right for your workload.



[README](#) [Code of conduct](#) [Contributing](#) [MIT license](#) [Security](#)

Microsoft Foundry — Model Router Auto Evaluation

New here? Start with the [QUICKSTART](#) for a no-keys-needed demo in under 2 minutes, then come back here for the full reference. If you prefer Jupyter, jump straight into [WALKTHROUGH.ipynb](#).

What is this?

Model Router is a Microsoft Foundry feature that automatically picks the cheapest model that can answer each prompt well. This toolkit answers the question every team asks before adopting it: "Will Model Router actually save me money on *my* workload without hurting quality?"

You plug in your prompts, point at your endpoints, and get a side-by-side report comparing Model Router to any baseline model on three things that matter: **quality, cost, and latency**.

May 19th, 2026 1 reaction

How to run evals for the model router

   Sanjeev, leestott, Marie-Louise

 1
 0
 1

One endpoint. Smarter spend. Model router in Foundry Models picks the optimal LLM for every prompt in real time based on signals like complexity, reasoning, and task type. Now with access to 28 frontier models, the model router makes model selection easier for developers and reduces manual overhead. This article walks you through how to run evaluations using a new open-source GitHub repo designed explicitly for the model router. [Access the Eval Repo on GitHub](#)

Before diving in, here are a few notes that developers should consider when building with the model router:

- The effective context window equals the smallest underlying model's window. Oversized prompts only succeed if the router happens to pick a model that can handle them.
- Claude models require separate deployment before the router can route to them.
- Routing decisions are text-only. Vision inputs are accepted, but images don't influence the routing decision and audio isn't supported.
- Available regions are currently East US 2 and Sweden Central, in Global Standard and Data Zone Standard deployment types.

Table of contents

Doesn't Foundry already have benchmarks? What does this do?

A step-by-step guide to running evals

Quick preview (no API keys needed)

Step 1 — Install

Step 2 — Set up credentials

Step 3 — Configure

Step 4 — Bring your prompts

Step 5 — Run the eval

Read: <https://devblogs.microsoft.com/foundry/how-to-run-evals-for-model-router/>

Reactor

MODEL RELEASES

How do I build my model optimization playbook in Foundry

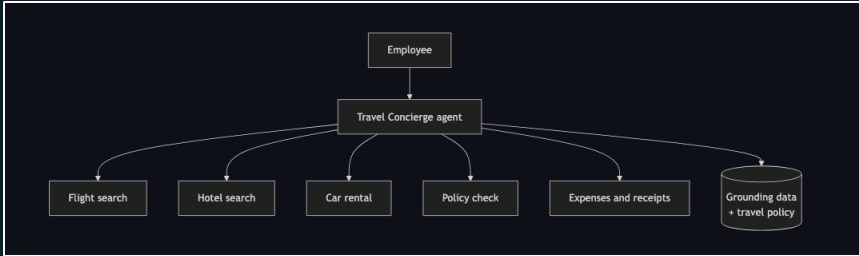
Nitya Narasimhan

<https://aka.ms/model-releases>



One Demo Scenario

Policy-compliant Travel Concierge



Employee : *"I need to fly to Berlin next Monday for a client meeting. Book my flight tickets, find me a hotel near Alexanderplatz, and make sure you stay within my company's travel policy and also check if you can expense my parking tickets, I have attached my parking receipts."*

What the AI system has to do



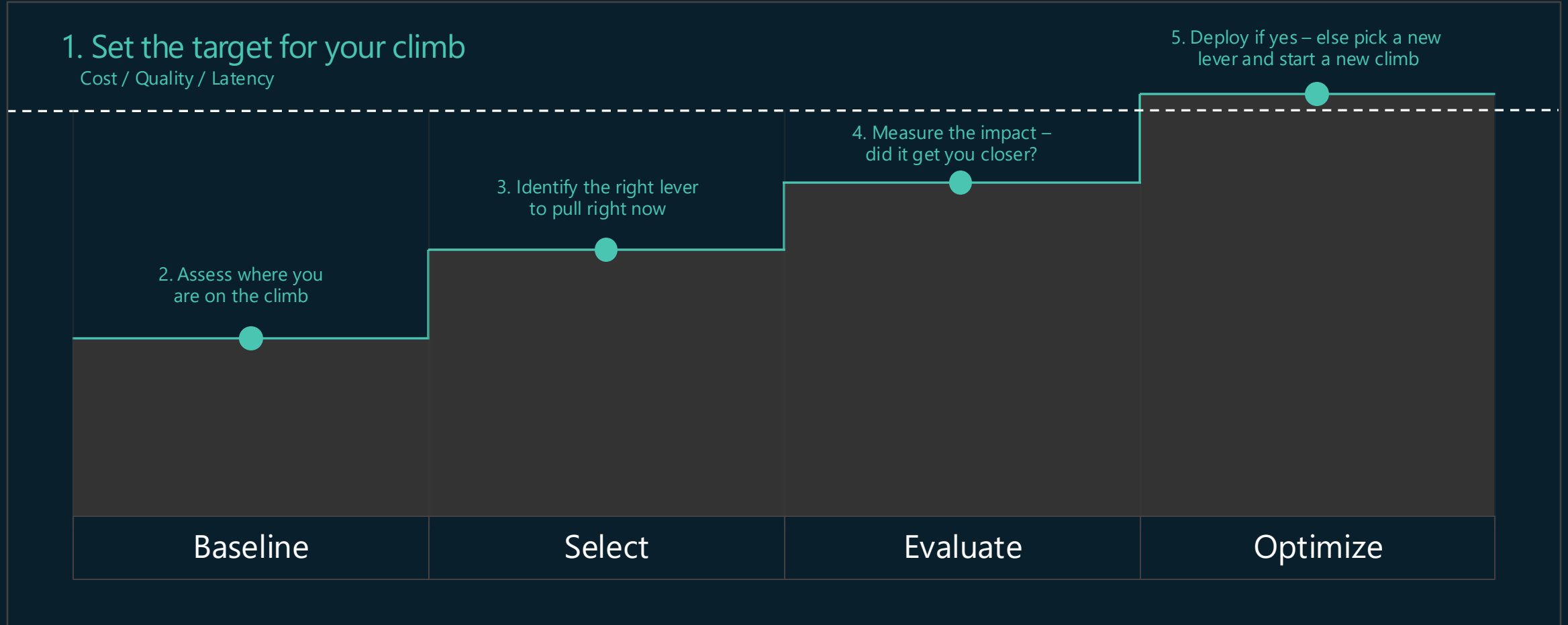
Success criteria

Correct The itinerary matches Carmen's request	Compliant Request adheres to the company travel policy	Usable The output provides user with clear and actionable guidance	Production-ready Response was fast, cost-effective and reliable
--	--	--	---



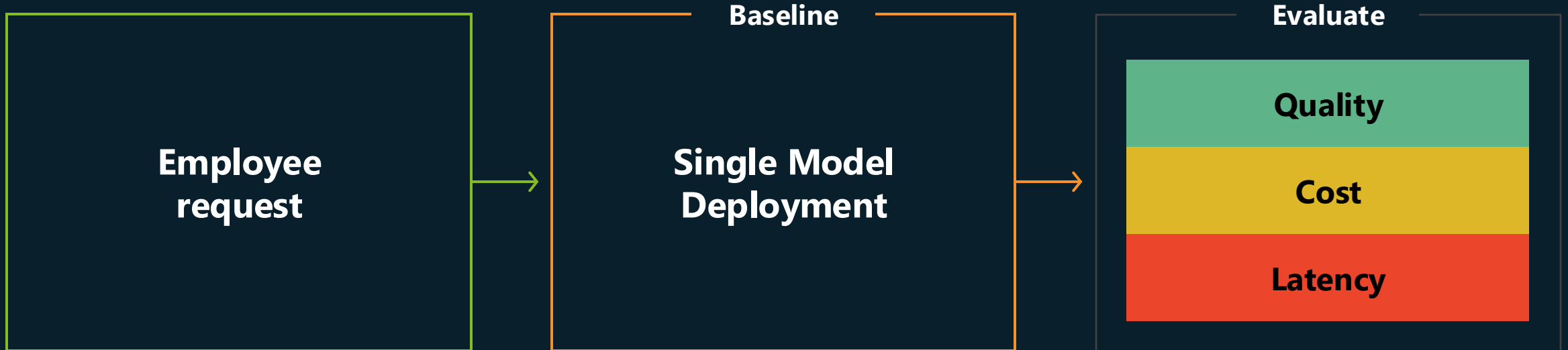
Make it Better / The Hill Climb

Incremental and measurable progress towards goals



Make it work / Deploy Baseline

One model, many jobs



Prototype fast with frontier model. Measure everything. Establish hill climbing targets.

Q1: How can we achieve **prompt optimization** with our frontier model?

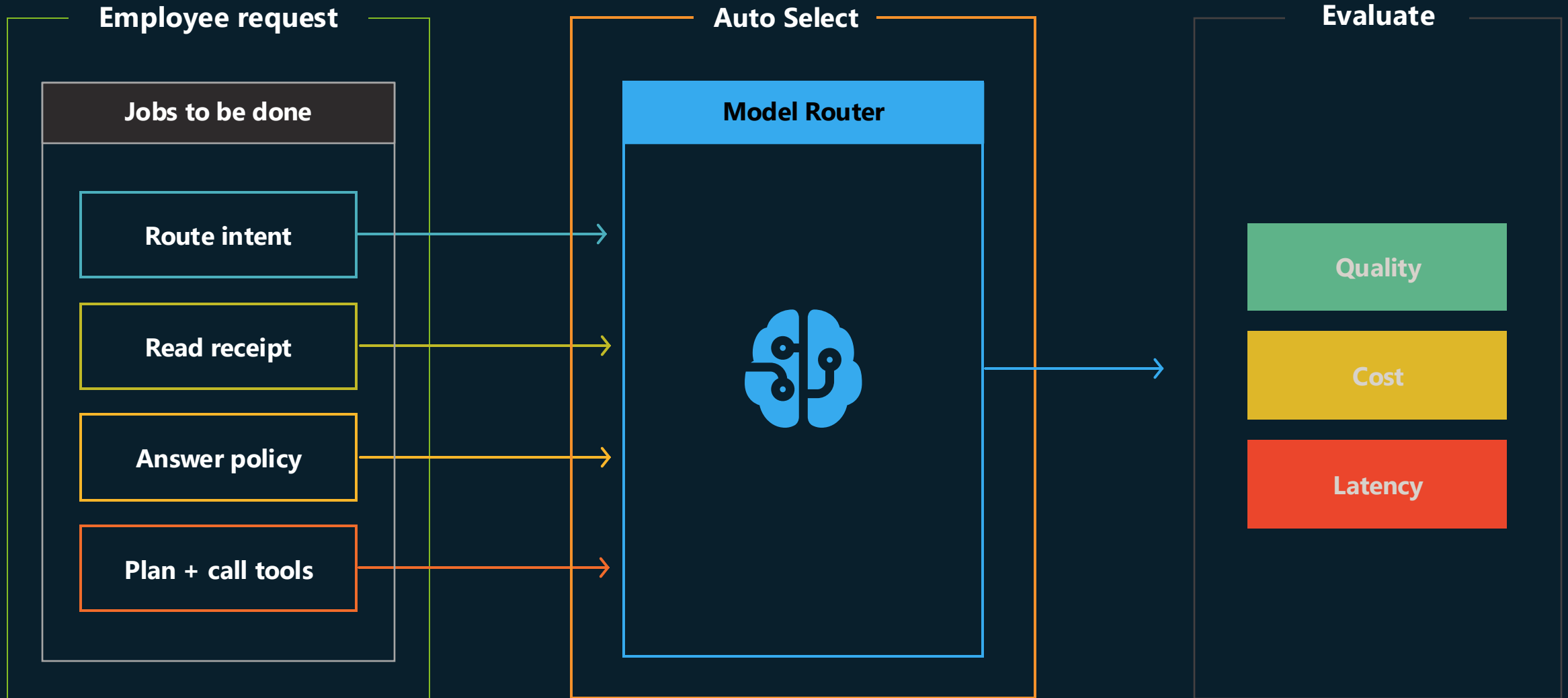


Q2: How can we use **model router** to optimize model selection?

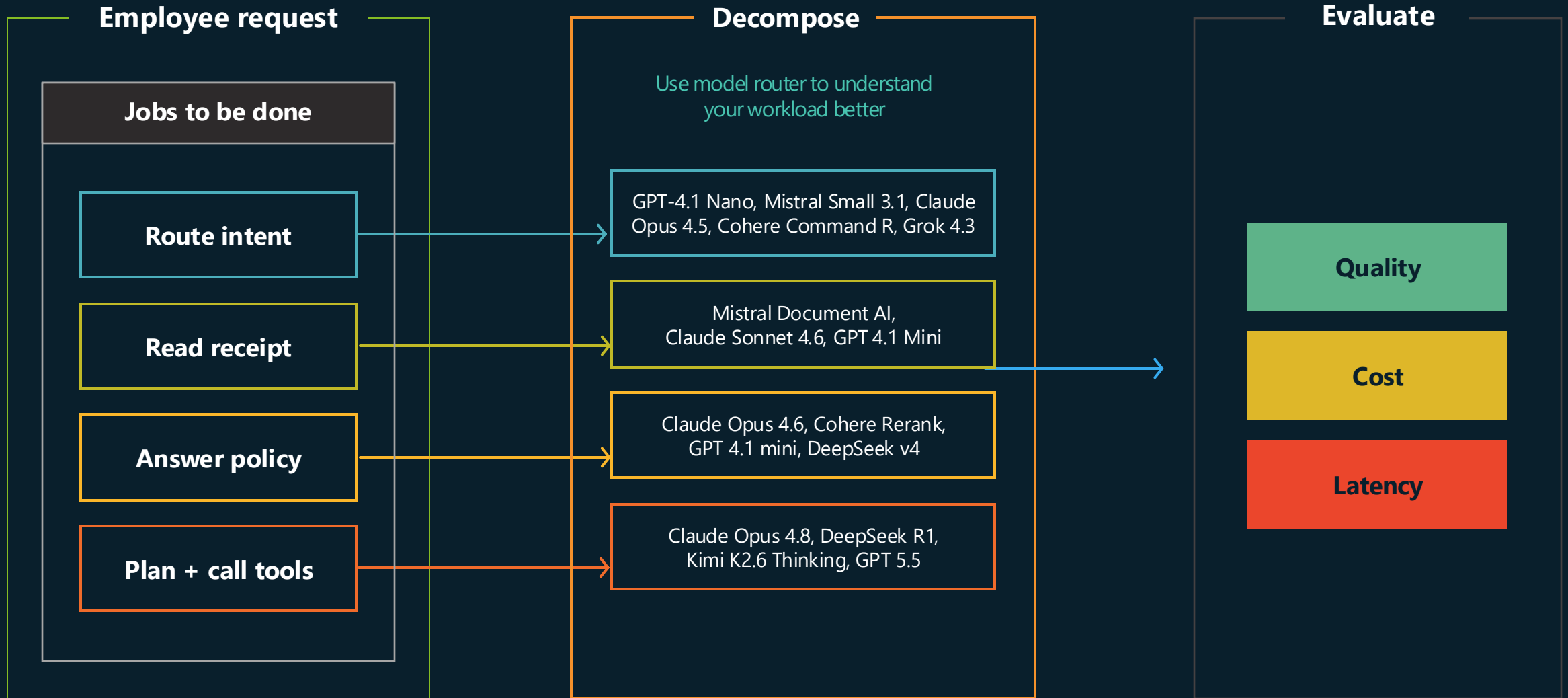


Step 1 / Replace frontier model with model router

One model, right-sized for the job

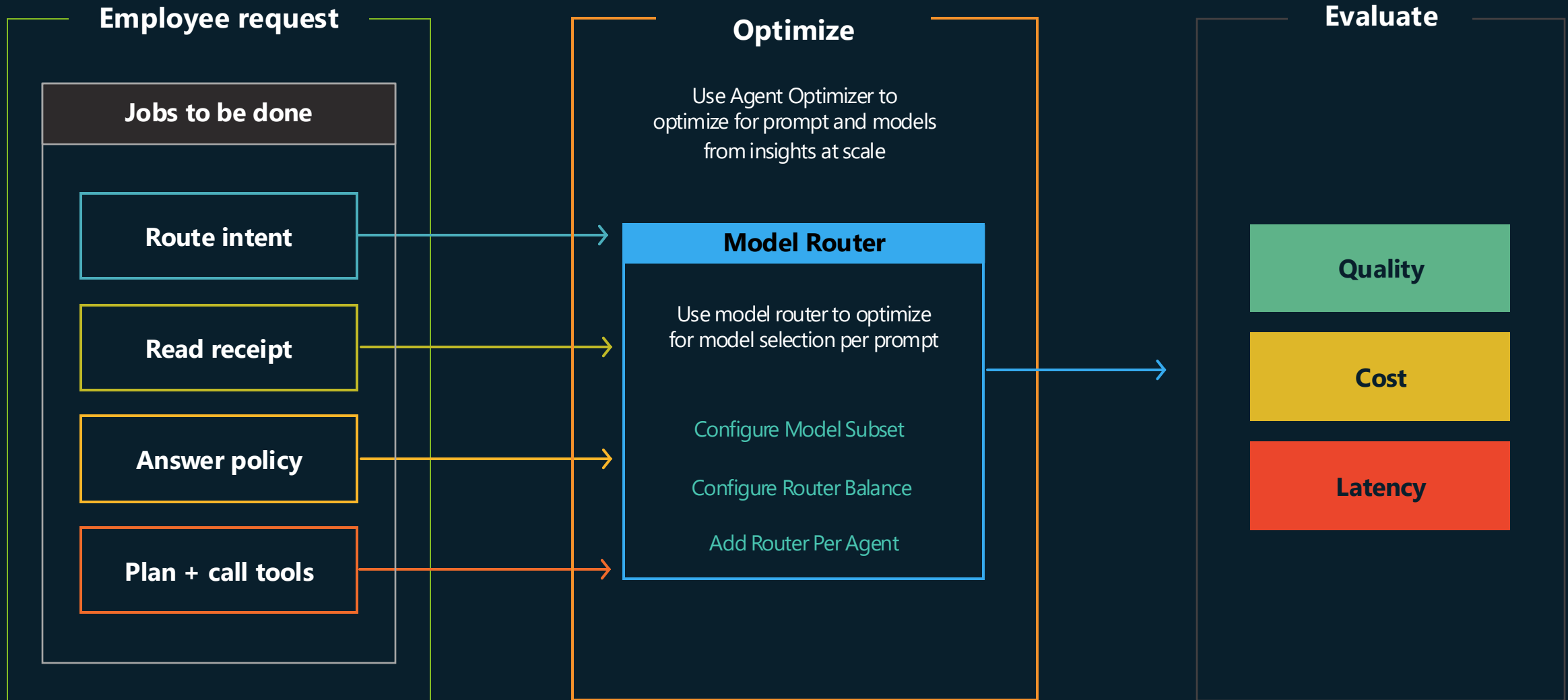


Many models, right-sized for the need



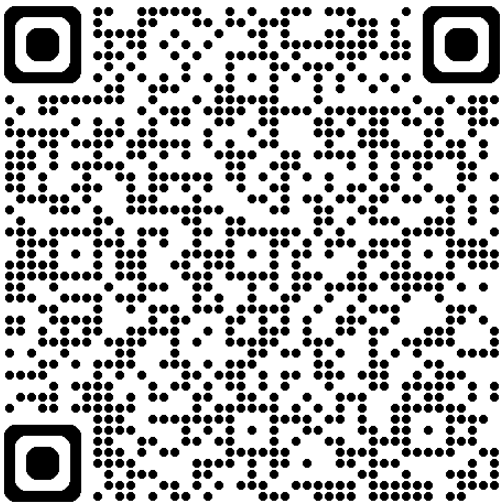
Step 3 / Decide: What's the right Foundry model for the task?

Many models, right-sized for the need



Explore Model Releases
Each Month to learn
model capabilities

Use a hill climbing
approach to see if new
features get you to target



Use the notebooks to explore various model router features – one at a time – with the Travel Concierge. Build your intuition for **router optimization** patterns

The notebook series

Each notebook opens with a falsifiable developer question — *does changing X improve Y?* — deploys a named router with one configuration change, runs the Contoso benchmark, and reads the scorecard to accept or reject the change. The lever in each notebook maps to the release that introduced it.

#	Notebook	Question: does changing X improve Y?	Lever (release)
00	00-setup.ipynb	Is our project, router, and agent ready before we optimize?	Self-contained setup
01	01-optimize-instructions.ipynb	Can we lift quality on a fixed frontier model just by optimizing its instructions?	Agent Optimizer + rubric evaluator (from code)
02	02-route-each-request.ipynb	Can one <code>model-router</code> endpoint serve the whole travel workload, and can we see which model answered?	Router as a model + <code>response.model</code> (2025 preview)
03	03-routing-modes.ipynb	Does Cost mode lower cost per request without dropping below the policy-accuracy gate — and does Quality mode lift accuracy enough to justify the spend?	Routing modes (GA 2025-11-18)
04	04-model-subset.ipynb	Does constraining the model subset hold quality while cutting cost and routing variance?	Custom model subset (GA 2025-11-18)
05	05-deployment-and-data-zone.ipynb	Does a Data Zone Standard deployment keep the frontier while meeting data-residency policy?	Deployment types (GA 2025-11-18)
06	06-automatic-failover.ipynb	Does automatic failover preserve success rate and tail latency when a model is unstable?	Automatic failover (Mar 2026)
07	07-agentic-routing.ipynb	Does routing the agent's tool-calling turns beat pinning one model, now that it selects across OpenAI, OSS, and Anthropic?	Agentic routing across providers (Aug 2026)
08	08-refresh-and-per-agent-routers.ipynb	Does giving each agent its own routing policy beat one global router, with the refreshed model pool?	Model refresh + per-agent routers (Aug 2026)

Notebooks 00-02 are built and runnable today. The later lever notebooks (03-08) roll out as fully runnable notebooks throughout August 2026 — the rows below are the planned sequence, and each becomes a live link as it ships.

5

Try It · Notebook Series

Weekly Highlights

WHAT'S NEW IN
MICROSOFT FOUNDRY?

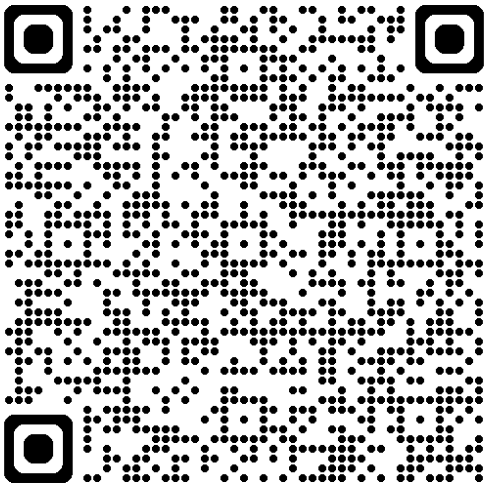
Sharmila Chockalingam

Director, AI Product Marketing @Microsoft



Open Model Choice

New DeepSeek and NVIDIA Nemotron Models



Many of today's most popular open models are advancing rapidly, particularly for agentic workflows, code generation, reasoning, document understanding, and workflow automation

options.

Today we're announcing several additions to the Foundry models catalog:

- **DeepSeek-V4-Flash-0731** as a Direct from Azure model and through Fireworks on Foundry
- **NVIDIA Nemotron 3.5 Lightning** through Fireworks on Foundry and through the Hugging Face collection on Foundry

More choices for agentic AI and enterprise applications

Many of today's most popular open models are advancing rapidly, particularly for agentic workflows, code generation, reasoning, document understanding, and workflow automation.

For example, **DeepSeek-V4-Flash-0731** introduces improvements in coding-agent performance, tool use, and workflow automation, making it well suited for AI assistants and agentic applications.

Similarly, **NVIDIA Nemotron 3.5 Lightning** is designed for agentic workloads with capabilities such as tool calling, long-context processing, multilingual support, structured outputs, and multi-step task execution.

One model, multiple deployment choices

As the model ecosystem grows, customers increasingly want flexibility in how they consume models.

With these launches, developers can access models through different pathways in Foundry depending on their requirements:

Claude GA

5 new tool capabilities with Claude in Microsoft Foundry

Structured outputs, Web search, Web fetch, MCP connector, and Tool search are now available for Claude models in Microsoft Foundry hosted on Azure

	Hosted on Azure	Hosted on Anthropic
Where inference runs	Anthropic-operated service on Azure infrastructure	Anthropic-operated service on Anthropic infrastructure
Model availability	Latest Opus, Sonnet, and Haiku models	The full Claude catalogue on Foundry
Deployment types	Global Standard US Data Zone Standard	
The five features in this post	✓	
Recommended for	Most workloads	

Copy

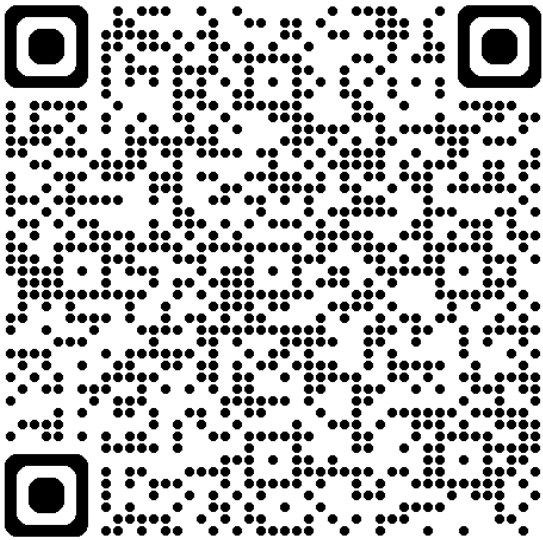
```

for attempt in range(3):
    raw = call_model(prompt)
    try:
        data = json.loads(raw)
        validate(data)
        break
    except (json.JSONDecodeError, ValidationError):
        prompt += "\n\nYour last response was invalid"

```

Safe Agents

Hands-on guide to build
safe agents in Foundry



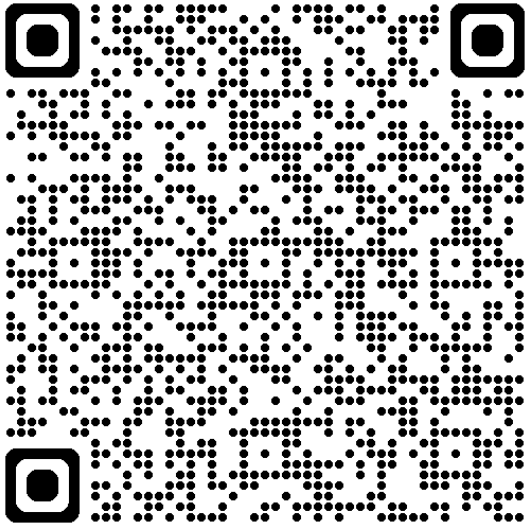
This hands-on guide turns the four SAFE principles into deterministic runtime controls for a hosted agent in Microsoft Foundry Agent Service using Agent Framework and ACS.



Visit: [Build a SAFE Agent on Microsoft Foundry | Microsoft Community Hub](#)

Real World
Development

Lessons learned while
building production
ready agents



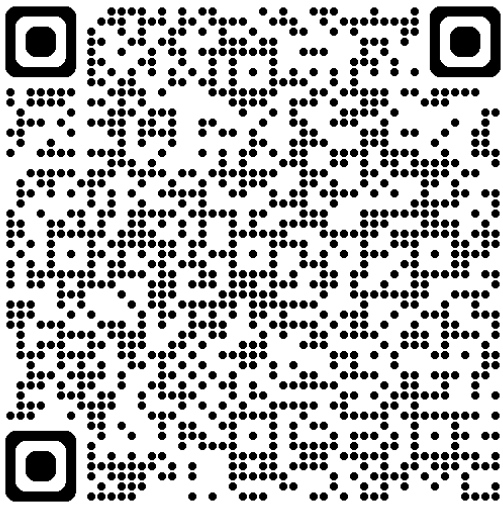
10 practical lessons learned from designing and evaluating AI agents with Microsoft Foundry, along with recommendations to help teams build reliable, secure, and scalable AI solutions



1 Read It · Lessons Learnt

Model Router Updates

New regions, refreshed
model pool and
understanding the hill
climb



Model router in Foundry Models brings the hill climb to the selection layer.

1 deployment, 1 endpoint, 27 models

NEW MODELS THIS MONTH

4 added 5 deprecated

OPENAI		DEEPSEEK	ANTHROPIC
gpt-4.0	gpt-5.2	Deepseek-V3.2	claude-haiku-4.5
gpt-4.0-mini			claude-sonnet-4.5
gpt-4.1	gpt-5.4-nano	META	
gpt-4.1-mini	gpt-5.4-mini	Llama-4-Maverick-FP8	claude-opus-4.6
gpt-4.1-nano	gpt-5.4	XAI	claude-opus-4.7
o4-mini	gpt-5.5	grok-4	
gpt-5-nano	gpt-oss-120b	grok-4-fast-reasoning	
gpt-5-mini			
gpt-5			



Reactor

MODEL MONDAYS S4:E04

From Research To Reality



Hosted By: Nitya Narasimhan

Aug 31, 2026 · 1:30-2:30 PM ET

Gulsimo
Osimi

Technical PM,
Core AI
Microsoft



Yash
Lara

PM Lead,
Microsoft
AI Research



WATCH THE
LIVESTREAM



Mon 1:30pm ET
Register Here

<https://aka.ms/model-mondays/RSVP>

JOIN THE
COMMUNITY



Fri 1:30pm ET
Join The Discord

<https://aka.ms/model-mondays/Discord>

STAR US ON
GITHUB



Explore Blogs,
Recaps & Schedules

<https://aka.ms/model-mondays>